

HOME BAKERS PLATFORM

Master AI Agent Completion Specification — Requirements Summary

Purpose: This document captures exactly what the Home Bakers platform must deliver across the three integrated applications (Customer App, Admin App, Backend + PostgreSQL). It is derived directly from the master specification PDF (*Home_Bakers_AI_Agent_Master_Task.pdf*, 14 pages) and the final verified implementation.

1. System Overview

Three integrated applications:

- **Customer App (Cakedeliveryapp)** — React Native: signup/login/catalog/cart/address/checkout/orders/tracking/notifications/reviews/profile
- **Admin App (cakedeliveryadmin)** — React Native: secure login/dashboard/catalog/orders/delivery tracking/store profile/onboarding
- **Backend (Backendofbakersapp)** — Node.js + Express + Prisma 7 + PostgreSQL: real transactional APIs, JWT auth, authorization, centralized error handling

2. Non-Negotiable Rules (From Master Spec)

- Work from the **existing codebase**; do not rebuild from scratch.
- Do not invent new API contracts when an endpoint/model already exists.
- Do not replace working features with mock implementations.
- Remove **fake/mock success** wherever real backend capability exists.
- Do **not** use hardcoded LAN/IP URLs (10.x.x.x, 192.168.x.x, localhost) in business code; use one centralized, environment-configurable API base URL per app.
- Never expose passwords, password hashes, JWT secrets, or sensitive server info.
- Never log plaintext passwords or JWT secrets.
- Protect admin endpoints with **real backend authorization** (customer token must not access admin mutation routes).
- Keep naming consistent across frontend, backend, and database (e.g., Category vs Categorys, AddressUI vs Addressscreen).
- Run safe **additive Prisma migrations** only; never destroy/reset production data.

3. Security & Authorization Requirements

- **JWT authentication** for both customer (/api/auth/signupmain, /loginmain) and admin (/api/auth/adminsignup, /loginadmin).
- **Real authorization middleware** (requireAuth, requireAdmin) on protected routes.
- Customer token must **not** access admin mutation endpoints (catalog create/edit/delete, analytics, orders mutation).
- Customer can only access **their own** cart, orders, notifications, profile, addresses.
- **Passwords hashed with bcrypt**; never returned in API responses.
- Validate IDs, quantities (> 0), prices, allowed order/payment statuses, and review ratings (1–5) server-side.
- Use **database transactions** for order creation (order + items + payment + stock decrement + cart clear).
- Prevent duplicate orders caused by repeated taps/network retries.

4. Phase-by-Phase Delivery Requirements

Phase 1 — Audit Before Coding

Inspect all three projects; identify package versions, entry points, navigation structure, API layers, controllers, routes, Prisma models, migrations. Build an internal feature/status matrix. Search for: hardcoded URLs/IPs, fake timeouts, mock data, missing imports, broken navigation targets, undefined fields, mismatched props.

Phase 2 — Backend Foundation

Reliable server startup; verified DATABASE_URL / .env; Prisma client generation and schema validity; centralized error handling; real JWT middleware; separate customer vs admin authorization roles; standardized API responses (`{ success, message, data?, error? }`); request body/param/query validation; CORS and environment variables configured.

Phase 3 — Authentication (Customer + Admin)

Customer: signup, login, logout, session persistence, password reset (if UI requires), change password.

Admin: admin login, protected session, logout, backend role enforcement.

Store only minimal session info; send access token consistently.

Phase 4 — Products / Catalog

Backend: list, get by ID, search, category filter, create/update/delete (admin), availability toggle, image upload, price/stock/prep time/weight validation, support existing flags (best seller, featured, public catalog).

Customer: real catalog from backend; real search/filter/category; real product details; real availability; real add-to-cart.

Admin: real catalog list; create/edit/delete; real image upload; real availability toggle; refresh after mutations.

Phase 5 — Cart (Persistent, User-Scoped)

Use `/api/cart` endpoints. Fetch current user's cart after login. Add/update/remove with correct quantities.

Prevent invalid quantities and out-of-stock products. Prevent unauthorized access to another user's cart. Persist to PostgreSQL.

Phase 6 — Address + Profile

Address CRUD (create/update/delete/select default) using real endpoints. Profile fetch/update/change-password. Never return password. Admin sees only authorized data.

Phase 7 — Real Checkout / Order Creation

Implement a **real transactional order flow**:

1. Authenticate customer
2. Load cart from DB
3. Load products from DB
4. Validate availability/stock
5. Recalculate subtotal server-side (never trust client total)
6. Apply delivery/tax/discount rules only if supported
7. Validate selected address
8. Create order + order items
9. Create/update payment record
0. Clear cart **only after successful order creation**
1. Return order ID, order number, status

2. Make retry behavior safe against duplicate orders

Phase 8 — Delivery Tracking

Store/retrieve tracking data through backend. Ensure tracking belongs to the requested order. Customer sees only their own tracking. Admin updates tracking; status and tracking stay logically consistent. No fabricated courier data.

Phase 9 — Payments

The existing `/api/upi/save` endpoint is a stub. If real payment gateway credentials/config are absent: **do not pretend a payment is successful**. Implement explicit states (Pending / Paid / Failed / Refunded) according to the database contract. If real gateway is required but credentials are missing, document the integration boundary clearly; do not hardcode secrets.

Phase 10 — Notifications + Reviews

Notifications: create when appropriate; list for correct user; mark individual notifications read; unread state must persist; do not fabricate unread counts.

Reviews: customer creates review only where permitted; validate rating (1–5); list by product; connect product detail UI to real reviews; prevent unauthorized identity spoofing; exclude password from user include.

Phase 11 — Admin Dashboard

Replace simulated/fallback chart values with **real aggregated backend data** from `/api/dashboard/analytics`. Show: total revenue, total orders, active orders, pending orders, delivered orders, total products, low stock, recent reviews, recent orders, best sellers, sales overview. Do not show fabricated fallback business values; show zero/empty state with a message when data is missing. Do not derive incorrect “today” values ignoring timezone.

Phase 12 — Admin Catalog (Complete)

List (search/filter/sort), create, edit, delete, availability toggle, image upload with validation, proper loading/error/empty states, optimistic/local updates only after successful backend mutation, correct product ID and field mappings.

Phase 13 — Navigation / UI Cleanup

Fix broken navigation and imports. Known items to verify: misspelled `Notificaton.js` import, broken Blog navigation used as profile/address route, Category vs Categorys, AddressUI vs AdressUI, MenuCard prop mismatch (missing price, bakingTime), any undefined navigation target, any unreachable screen. Preserve existing visual identity; do not redesign unnecessarily.

Phase 14 — Network Configuration

Create centralized API configuration (`src/config/api.js`) for both customer and admin apps. Support: development (Android emulator), physical Android device, production/deployment. **No hardcoded** `10.x.x.x`, `192.168.x.x`, `localhost`, or other machine-specific URLs inside business screens/components. Document exactly where the developer changes the API URL.

Phase 15 — Error Handling + Loading States

Every critical screen must handle: loading, success, failure, timeout, network error, unauthorized, server error, empty response. No raw internal errors exposed to clients; centralized backend error handling preferred.

Phase 16 — Database / Prisma Reconciliation

Reconcile Prisma schema names/relations with actual database tables. Verify: User, Address, Cart, Product, Order, OrderItem, Payment, Review, Notifications, DeliveryTracking, StoreProfile. Ensure relations: User-Address, User-Cart, Product-Cart, Order-OrderItem-Product, Order-Payment, Order-DeliveryTracking. Add safe additive migrations; never rename tables/columns destructively; never reset production-like data.

Phase 17 — Build / Lint / Error Cleanup

Run `npm install`, Prisma validation/generation/migrations, backend startup, Metro build, lint/test checks. Fix compile errors, import errors, navigation errors, Prisma errors, API 4xx/5xx caused by implementation, undefined fields, incorrect response parsing. Fix all warnings that indicate real bugs.

Phase 18 — Final Verification

Search again for hardcoded API URLs, fake `setTimeout` checkout/success, mock product/order arrays in production paths, unprotected admin routes, password fields returned/logged, inconsistent order status strings, broken navigation names. Verify customer/admin/backend contracts match. Verify database relations and migrations. Verify fresh install/startup works with documented commands. Produce a final completion report.

5. Testing Requirements (Cross-App)

Customer end-to-end: signup → login → home → search/category → product detail → add to cart → cart update → address → checkout → order success → order history → tracking → notification → profile.

Admin end-to-end: admin login → dashboard → catalog → add/edit/delete/availability/image → orders → order detail → status update → tracking → dashboard refresh.

Cross-app test: customer creates order → DB receives order → admin sees order → admin updates status → customer sees updated status → admin updates tracking → customer sees tracking → notifications created/read correctly.

Negative/security tests: invalid login, duplicate signup, expired/invalid token, customer calling admin endpoint, customer requesting another customer's order, invalid product ID, quantity 0/negative, out-of-stock purchase, invalid order status, malformed address, failed network request, duplicate checkout tap, missing image/file, invalid review rating.

6. Definition of Done

The project is complete **only when**:

- The three codebases behave as **one coherent application**.
- Customer actions **persist to the database**; admin actions affect the **same database state**.
- API contracts match between frontend expectations and backend responses.
- Authentication/authorization is enforced end-to-end.
- Orders and statuses are consistent across DB/backend/admin/customer.
- **No critical flow relies on fake/mock success.**
- No hardcoded developer LAN IP remains in business/API code.
- No password or secret is logged or returned.
- All critical navigation targets exist and work.
- Backend and both apps can be started using documented commands.
- Final build/run/test checks pass, or any unavoidable limitation (external service credentials, mobile build environment) is explicitly documented.

7. Acceptance Checklist (Extracted from Spec)

Check	Requirement
✓	Customer signup/login works against backend; survives app restart.
✓	Admin login works; admin-only resources reject customer credentials.
✓	Catalog is database-driven in both apps.
✓	Product create/edit/delete/availability/image upload works end-to-end.
✓	Cart persisted in PostgreSQL; scoped to authenticated customer.
✓	Address CRUD and default selection work.
✓	Checkout creates a real database order ; clears cart only on success.
✓	Server recalculates order totals; does not trust client-provided total.
✓	Customer sees their own orders only.
✓	Admin sees and updates orders; status updates reflected to customer.
✓	Order status values consistent: Pending, Accepted, Preparing, Ready, Delivered, Cancelled, Rejected.
✓	Tracking shared correctly between admin updates and customer view.
✓	Payment state never falsely reported as Paid (stub boundary documented).
✓	Notifications persist; read state works.
✓	Reviews use authenticated identity and validated ratings.
✓	Dashboard numbers come from real data; no fabricated fallback values.
✓	No hardcoded developer LAN IP in business/API code.
✓	No password or secret logged or returned.
✓	All critical navigation targets exist and work.
✓	Apps start with documented commands.

8. Run Commands (Documented)

Customer App:

```
cd Cakedeliveryapp && npm install && npm start (or npx react-native run-android)
```

Admin App:

```
cd cakedeliveryadmin && npm install && npm start (or npx react-native run-android)
```

Backend:

```
cd Backendofbakersapp && npm install && npm start (port 3000 by default)
```

API URL configuration: Change the base URL in Cakedeliveryapp/src/config/api.js and cakedeliveryadmin/src/config/api.js for physical device / production.

9. Final Verification Requirements

Before declaring complete:

1. Search again for any remaining hardcoded API URLs.
2. Search again for any fake setTimeout checkout/success behavior.
3. Search again for mock product/order arrays in production paths.
4. Search for unprotected admin routes.

5. Search for password fields returned/logged.
6. Search for inconsistent order status strings.
7. Search for broken navigation names or unreachable screens.
8. Search for undefined navigation targets.
9. Verify customer/admin/backend contracts align.
0. Verify database relations and safe migrations.
1. Verify fresh startup works.
2. Document any remaining limitation explicitly (e.g., real payment gateway requires external credentials; mobile APK requires native build environment).